UNIVERSIDAD DE LA AMAZONIA · ANÁLISIS DE SISTEMAS

Cómo desarrollé el juego

La idea, las herramientas, las gráficas, las animaciones y el código

Código: **github.com/ju-pelaez/arbol-problemas-pasaporte-verde**
Sitio: **arbol-problemas-pasaporte-verde.vercel.app**



Juan Pablo Peláez Pino
Ingeniería de Sistemas

Proyecto Pasaporte Verde
Octubre de 2026

1 · LA IDEA

Por qué hice un juego

Un árbol de problemas se entiende de verdad cuando uno lo construye. Por eso, en vez de solo mostrar la diapositiva, hice un juego donde el árbol se descubre a partir de una historia y se arma pieza por pieza.

Partí del análisis que hicimos para el proyecto **Pasaporte Verde del Predio Amazónico**: el pequeño productor del Caquetá no puede demostrar que su producción es legal y libre de deforestación, porque los actores, los datos, los procesos y los mecanismos de control del sistema no están articulados. El juego tenía que enseñar eso sin proponer soluciones, porque la materia es análisis.

Los requisitos que me puse

Como es una materia de análisis, empecé por los requisitos, igual que con cualquier sistema:

Tipo	Requisito
Funcional	Explicar las partes del árbol (suelo, raíces, tronco, copa) y las 4 columnas de causas.
Funcional	Contar el problema con una historia: el lote de cacao de Don Arley (personaje inventado) que rechazan en Europa.
Funcional	Usar los actores reales: ICA, IDEAM, CORPOAMAZONIA, ANT, IGAC, gremios, Unión Europea.
Funcional	Que todo el salón juegue a la vez desde el celular, con nombres, puntos y equipos.
No funcional	Funcionar en cualquier navegador, sin instalar nada, en celulares baratos.
No funcional	Aguantar un salón grande (lo probé con 49 jugadores) y que nadie pierda sus puntos si se le cae la conexión.
No funcional	Funcionar aunque falle el internet (modo sin celulares).



La misión de Actores: cada actor real con su función, leída en voz alta.

2 · EL PROCESO

Cómo lo fui construyendo

No salió de una vez. Lo hice por versiones, probando cada una antes de seguir.

Versión	Qué agregué	Por qué
1	El primer juego del árbol	Misiones con preguntas para cada parte del árbol. Es el que le mostré a la profe.
2	Los capítulos de Don Arley	Para que las causas salgan de una historia: el lote viaja, lo rechazan y las fallas del camino se vuelven raíces.
3	El mapa para caminar y los minijuegos de movimiento	Para que sea un videojuego de verdad: carrera de portales, lanzamiento con resortera, lluvia de tarjetas.
4	La voz que lee e ilumina	Para que la explicación se escuche y se siga con los ojos al mismo tiempo.
5	Los actores reales	La profe pidió nombres reales: investigué quién es quién y qué función cumple cada uno.
6	El Reto del salón	Que todos jueguen desde el celular: QR, nombres, puntos, equipos, capítulos y juegos de unir, ordenar y clasificar.
7	Las animaciones	Que cada momento importante se sienta: el árbol que se dibuja, la cuenta regresiva, el podio y el confeti.
8	Enlazar y armar el árbol	Cambié la carrera por juegos más claros: tender un hilo desde cada tarjeta hasta su grupo, y arrastrar cada pieza a su parte del árbol (copa, tronco, raíces).

Las aplicaciones y herramientas que usé

Herramienta	Para qué
Editor de código	Escribir el HTML, el CSS y el JavaScript.
Navegador (Edge / Chrome) y sus herramientas de desarrollador	Probar el juego, ver errores en la consola y medir que las animaciones no pesen.
Node.js	Revisar que el código no tenga errores de sintaxis.
Python	Correr un servidor local para probar y generar la presentación del árbol con la librería `python-pptx` .
PowerPoint	La presentación del árbol y el mapa de actores.
Git y GitHub	Guardar las versiones del código.
Vercel	Publicar el juego en internet con un enlace.
PeerJS y **qrcode-generator**	Dos librerías gratuitas: conectar los celulares y dibujar el QR.
Google Fonts (Nunito)	La única letra de todo el juego.

3 · LAS GRÁFICAS

Cómo hice las gráficas

El juego no usa ninguna imagen: todo lo que se ve está dibujado con código. Por eso pesa muy poco y carga rápido hasta en un celular viejo.

Con qué	Qué dibujé
Canvas 2D	El mundo para caminar (cielo, luna, luciérnagas, árboles, camino) y los minijuegos de movimiento. Se redibuja unas 60 veces por segundo. También el confeti del final.
SVG	Los árboles de problemas: las raíces son curvas (`path`), la copa son círculos y el suelo un rectángulo. Al ser vectores se ven nítidos en el proyector.
CSS	Las tarjetas, botones y paneles, con bordes gruesos y sombras sólidas para que parezcan de cartón.
Emojis	Los íconos (🌳 🍫 👥 📊 🔁 🎯) y los avatares de los jugadores: se ven en cualquier celular sin descargar nada.

Los colores

Cada componente tiene siempre el mismo color en todo el juego, para que el ojo los reconozca sin leer:

Color	Código	Uso
Verde	`#1E8C4E`	Actores
Azul	`#2B7BB9`	Datos
Naranja	`#D0661C`	Procesos
Dorado	`#C99300`	Control
Verde selva	`#0E3424`	Fondo (la selva de noche)
Amarillo	`#FFC23D`	Lo importante: botones, puntos, títulos

La letra

Todo usa **Nunito**, una letra redondeada que se lee bien de lejos en el proyector y se siente amigable.

4 · LAS ANIMACIONES

Cómo pensé las animaciones

Seguí principios de diseño de movimiento: cada animación tiene que servir para algo (avisar, orientar o celebrar), no solo adornar.

Mis 5 reglas

1. **Lo que pasa pocas veces puede lucirse; lo que se toca muchas veces, no.** El final y los capítulos tienen animaciones grandes. Los botones del celular responden al instante, porque se tocan muchas veces.
2. **Entrar = aparecer + subir un poquito + enfocarse.** Los elementos llegan desde un desenfoque (`blur`) a nítido: se siente como si se materializaran.
3. **Curvas propias, nunca la de por defecto.** Uso una curva que llega rápido y frena suave, y un "resorte" hecho con la función `linear()` de CSS para lo que rebota.
4. **Solo animo `transform` y `opacity`.** Así la tarjeta de video hace el trabajo y no se traba en el computador del salón (las barras crecen con `scaleX`, no cambiando el ancho).
5. **Respeto a quien prefiere menos movimiento.** Si el sistema tiene activada esa opción (`prefers-reduced-motion`), todo aparece sin animar.

Las animaciones del Reto del salón

Momento	Qué hace
Cambio de pantalla	La pantalla vieja se desenfoca y la nueva entra enfocándose (View Transitions API).
Alguien entra a la sala	Su nombre cae con un pequeño rebote en su equipo.
Capítulo nuevo	"CAPÍTULO 2" se descubre con un barrido, el ícono gira con resorte y el título aparece palabra por palabra.
Cuenta regresiva	Cada número llega grande y desenfocado, se enfoca, y un anillo se expande.
Responder	El círculo del jugador se enciende en el muro.
Revelar	La respuesta correcta crece y le pasa un brillo; las otras se apagan; las barras de votos crecen.
La pieza crece	Las raíces se **dibujan** de arriba abajo; la copa brota con resorte; salen 14 chispas.
Posiciones	Las filas llegan en cascada, las barras crecen y los puntos suben contando.
Final	El árbol se arma solo de abajo hacia arriba, sale confeti disparado desde las esquinas, el podio sube 3.º, 2.º y 1.º, y aparecen los premios.
Reacciones	Los emojis suben por el proyector y se mecen (dos movimientos separados: subir y balancearse).

5 · CÓMO FUNCIONA POR DENTRO

La arquitectura

Todo es HTML, CSS y JavaScript, sin frameworks. Es un sitio estático: no tiene servidor propio ni base de datos.

```
arbol-problemas-pasaporte-verde/
├── index.html      ← la expedición: mapa, misiones, minijuegos, voz
├── vivo/           ← los capítulos de la historia de Don Arley
├── salon/          ← el Reto del salón
│   ├── index.html · salon.css
│   ├── retos.js    ← el contenido: fallas, retos, capítulos
│   └── salon.js    ← el proyector, el celular y la conexión
└── docs/           ← el árbol, el mapa de actores y estas guías
```

Cómo se conectan los celulares

Proyector abre la sala "arbolvivo-salon-CÓDIGO" ⇄ **Servidor de PeerJS (solo para encontrarse)** ⇄

Celular entra con el código del QR

Uso **WebRTC** con la librería **PeerJS**. El servidor de PeerJS solo sirve para que el proyector y el celular se encuentren; después se hablan directo, con mensajes pequeños:

Mensaje	Quién lo manda	Para qué
`hola`	Celular	"Soy este jugador" (nombre + un id guardado en el celular).
`bienvenido`	Proyector	Le asigna avatar y equipo.
`capitulo` · `cuenta`	Proyector	Muestra el capítulo y el "¡Prepárate!".
`pregunta`	Proyector	El reto: opciones, fichas para ordenar o parejas para unir.
`resp` · `avance`	Celular	La respuesta (o lo que lleva en la lluvia de fallas).
`resultado` · `fin`	Proyector	Puntos, puesto, equipo y la explicación.
`reac`	Celular	Una reacción (máximo una cada 1,2 s por persona).

Que nadie pierda sus puntos

Cada celular inventa un **id** y lo guarda en el navegador (`localStorage`). Si se le cae la conexión o recarga la página, vuelve a mandar `hola` con el mismo id: el proyector lo reconoce, le devuelve sus puntos y le reenvía la pantalla donde iba.

6 · EL CÓDIGO CLAVE

Lo que tengo que saber explicar

1. El ciclo del juego (expedición)

Cada cuadro calculo cuánto tiempo pasó y muevo todo según ese tiempo. Así se mueve igual en un computador rápido o lento.

```
const dt = Math.min(.05, (ahora - antes) / 1000);  // segundos desde el cuadro anterior
carta.y += carta.vy * dt;                          // velocidad × tiempo
```

2. Cuánto acertó cada uno (Reto del salón)

Para escoger una opción, acertar vale 1. Para unir, ordenar o clasificar comparo ficha por ficha: 3 de 4 bien vale 0,75.

```
const fraccion = v => juego
  ? r.correctas.reduce((a, c, n) => a + (v.sel[n] === c ? 1 : 0), 0) / r.correctas.length
  : (v.k === r.correcta ? 1 : 0);
```

3. Los puntos

```
gano = (500 * f + (perfecto ? 500 * rapidez + 100 * (racha - 1) : 0)) * (ultimo ? 2 : 1)
```

4. Los equipos justos

El que entra va al equipo con menos gente, y el puntaje del equipo es el promedio, no la suma.

```
const equipoMenor = () => { const n = [0, 0, 0, 0]; jugadores.forEach(j => n[j.equipo]++); return
n.indexOf(Math.min(...n)); };
```

5. La raíz que se dibuja

A cada raíz le digo que mide 1 (`pathLength="1"`). Luego la "escondo" corriendo el trazo una longitud entera y lo devuelvo a 0: parece que se dibuja.

```
.pz.raiz.viva.nueva { animation: dibuja 1.1s var(--sale) both; }
@keyframes dibuja { from { stroke-dashoffset: 1; } }
```

6. La voz que ilumina

Parto cada explicación en bloques y pongo una frase por bloque en la cola de `speechSynthesis`. Cuando empieza cada frase, ilumino su bloque. Elijo primero las voces más naturales y quito los emojis antes de leer.

7 · PRUEBAS

Cómo lo probé

Prueba	Resultado
Un salón de **49 jugadores** (48 celulares simulados + uno real) jugando la partida completa de 11 retos	Todos recibieron los 11 retos y el final, sin errores.
Recargar la página de un celular en medio de un reto	Volvió solo, con el mismo nombre, equipo y puntos, y siguió en el reto.
Doble clic en "Seguir" durante una transición	No se salta ningún reto.
Cada tipo de juego (escoger, clasificar, ordenar, unir) desde el celular	Puntos parciales correctos y la solución con el porcentaje del salón.
Las 11 misiones y los 5 capítulos de la expedición	Funcionan de principio a fin.

Dónde está cada cosa en el código

Qué	Archivo y línea
Fallas, retos y capítulos	`salon/retos.js · FALLAS_SALON 20 · armarRetos 61 · CAPITULOS_SALON 96`
Equipos, reacciones, servidores STUN	`salon/salon.js · EQUIPOS 21 · REACCIONES 27 · ICE 29`
Transiciones, números que cuentan, chispas, confeti	`salon.js · transicion 54 · contar 63 · chispas 76 · confeti 97`
El árbol del salón (SVG)	`salon.js · arbolSVG 151 · pintarPieza 164`
Proyector: sala, jugadores, mensajes	`salon.js · anfitrion 174 · abrirPeer 199 · recibir 228 · pintarSala 282`
Capítulos, cuenta regresiva, reto, revelar	`salon.js · capitulo 343 · cuenta 369 · pregunta 409 · revelar 457`
Posiciones y final	`salon.js · posiciones 521 · final 542`
Celular: entrar, reconectar, responder	`salon.js · jugador 598 · conectar 647 · reintentar 681 · recibir 696`
Juegos del celular	`salon.js · jugarClasifica 762 · jugarOrdena 782 · jugarUne 798`
Animaciones	`salon/salon.css · sección "MOVIMIENTO" al final`

8 · PREGUNTAS

Preguntas técnicas que te pueden hacer

¿En qué lo programaste?

> HTML, CSS y JavaScript sin frameworks. Canvas para los juegos, SVG para los árboles, PeerJS para los celulares y Vercel para publicarlo.

¿Cómo se conectan 50 celulares sin un servidor?

> Con WebRTC: el proyector y cada celular se hablan directo. El servidor de PeerJS solo los presenta. Cada mensaje es un objeto pequeño, así que 50 conexiones no pesan.

¿Qué pasa si alguien recarga la página?

> Su celular guarda un id. Al volver, el proyector lo reconoce y le devuelve sus puntos y la pantalla donde iba.

¿Por qué el equipo gana por promedio?

> Para que no gane el equipo con más gente. Así cuenta que todos aprendan, no cuántos son.

¿Cómo hiciste que las animaciones no se traben?

> Solo animo transform y opacity, que los hace la tarjeta de video. Las barras crecen con scaleX en vez de cambiar el ancho.

¿Cómo funciona la voz?

> Con la Web Speech API del navegador: leo por bloques e ilumino el que está sonando.

¿Guarda datos de los compañeros?

> No. Los nombres solo viven en la página del proyector mientras dura la partida.

> **Para que sea tuyo de verdad, antes de la clase cambia algo tú mismo:** una falla en `salon/retos.js`, el tiempo por reto (`const TIEMPO = 20`) en `salon/salon.js`, o una frase de un capítulo. Luego publica con `npx vercel deploy --prod` .